PAYMENT PLATFORM / ARCHITECTURE & ENGINEERING

White-Label B2B2B Payment Gateway

Architecture blueprint and engineering roadmap

Build a platform-owned API and domain layer around Hyperswitch, while retaining Hyperswitch as the payment execution engine.

For licensed Payment Facilitators and Aggregators onboarding their own merchants and distributing embedded Android SDKs.

PLATFORM	Rust core / React and ReScript portal / Android SDK
DESIGN PRIORITIES	Tenant isolation, payment safety, terminal identity, API stability
MIGRATION PATH	Decoupled services with a future Java/Kotlin transition
REVIEW BASELINE	Core commit c5ecb434b419 / v1 feature baseline
DATE	22 October 2026

Includes the transaction architecture, domain mapping, PostgreSQL schema draft, onboarding API contract, SDK plan and phased delivery gates.

Contents

Architectural recommendation . . . 3

1. Terminology and domain mapping . . . 3

2. Architecture and service boundaries . . . 5

3. Aggregator Portal and access control . . . 7

4. PostgreSQL tenancy and pooled connections . . . 8

5. Terminal identity and transaction routing . . . 10

6. Distributed idempotency and payment safety . . . 11

7. Three-tier credential hierarchy . . . 13

8. Android SDK white-labeling and APK integration . . . 14

9. Database schema draft . . . 16

10. API architecture contract . . . 20

11. Engineering roadmap and production gates . . . 23

Reference index . . . 25

Source notes

The analysis uses core commit `c5ecb434b419d2c473cf2882a3866de9146abdec` and upstream dashboard and Android sources inspected for the architectural review. Core references link to that pinned commit. Some upstream main-branch references can change over time.

The schema and API examples are architectural drafts. They describe the proposed platform contract and require implementation, migration validation and the applicable terminal/acquirer approval process.

Architectural recommendation

Build a platform-owned API and domain layer around Hyperswitch, while retaining Hyperswitch as the payment execution engine. This gives you a controlled B2B2B model now and a practical path to Kotlin later.

Your proposed entity mapping is workable, with two qualifications: a `BusinessProfile` needs a separate merchant lifecycle and isolation model, and terminal identity/configuration must be modeled separately from connector credentials.

This blueprint is grounded in the local core checkout, commit `c5ecb434b419d2c473cf2882a3866de9146abdec`, and the upstream dashboard and Android repositories inspected for the architectural review. The core currently defaults to the `v1` feature set; pin compatible versions across all forks before implementation.

1. Terminology and domain mapping

Map business terminology through an explicit compatibility layer. Avoid globally renaming Hyperswitch tables, Rust types, and API fields. Give your public API the terminology your customers understand, then translate it into native Hyperswitch identifiers internally.

Your business entity	Hyperswitch representation	Required platform additions
Core platform	One or more infrastructure `Tenant` configurations	Platform operator IAM, deployment registry, global operational views
Aggregator / licensed PayFac	`MerchantAccount`	Aggregator lifecycle, commercial configuration, operational permissions, quotas
Aggregator administrative boundary	Prefer one Hyperswitch `Organization` per aggregator	Explicit organization-to-aggregator mapping
Aggregator's merchant	Canonical platform `Merchant`, initially mapped to a `BusinessProfile`	KYB/KYC references, underwriting status, acquirer enrollment, settlement configuration
Acquirer integration	`Connector` implementation	Protocol adapter, capabilities, certification requirements
Configured processing account	`MerchantConnectorAccount`	Mapping to an aggregator-owned acquirer account and merchant processing configuration
Physical terminal deployment	New terminal domain	Device identity, merchant assignment, app identity, activation and revocation
Terminal-to-processing relationship	New terminal binding domain	MID, TID, eligible MCA, certification bundle, encryption domain and configuration version

Three distinctions matter:

- **A Tenant is an infrastructure partition, not a super-admin identity.** In this checkout, tenant configuration includes PostgreSQL schemas, Redis prefixes and a ClickHouse database. Global visibility must come from a separately authorized platform control plane. See the tenant configuration model.

- **A BusinessProfile is a processing configuration boundary, not a complete legal sub-merchant account.** It belongs to a MerchantAccount and carries routing and payment configuration. Your merchant service must supply the missing lifecycle. See the profile model.
- **A Connector is code; an MCA is configured access to that connector.** A terminal has an independent lifecycle and may have several eligible processing bindings. See the MCA model.

A translation should therefore look like:

CONTRACT / FLOW

```
Public domain:
  aggregator_id = agg_A
  merchant_id   = mer_B
  terminal_id   = term_C

Internal mapping:
  agg_A -> Hyperswitch tenant T / organization O / MerchantAccount M
  mer_B -> BusinessProfile P under M
  term_C -> active deployment under mer_B
  binding -> MCA C under M/P, acquirer MID/TID, certification version
```

Public payloads should never require customers to understand that their aggregator is internally called a Hyperswitch merchant.

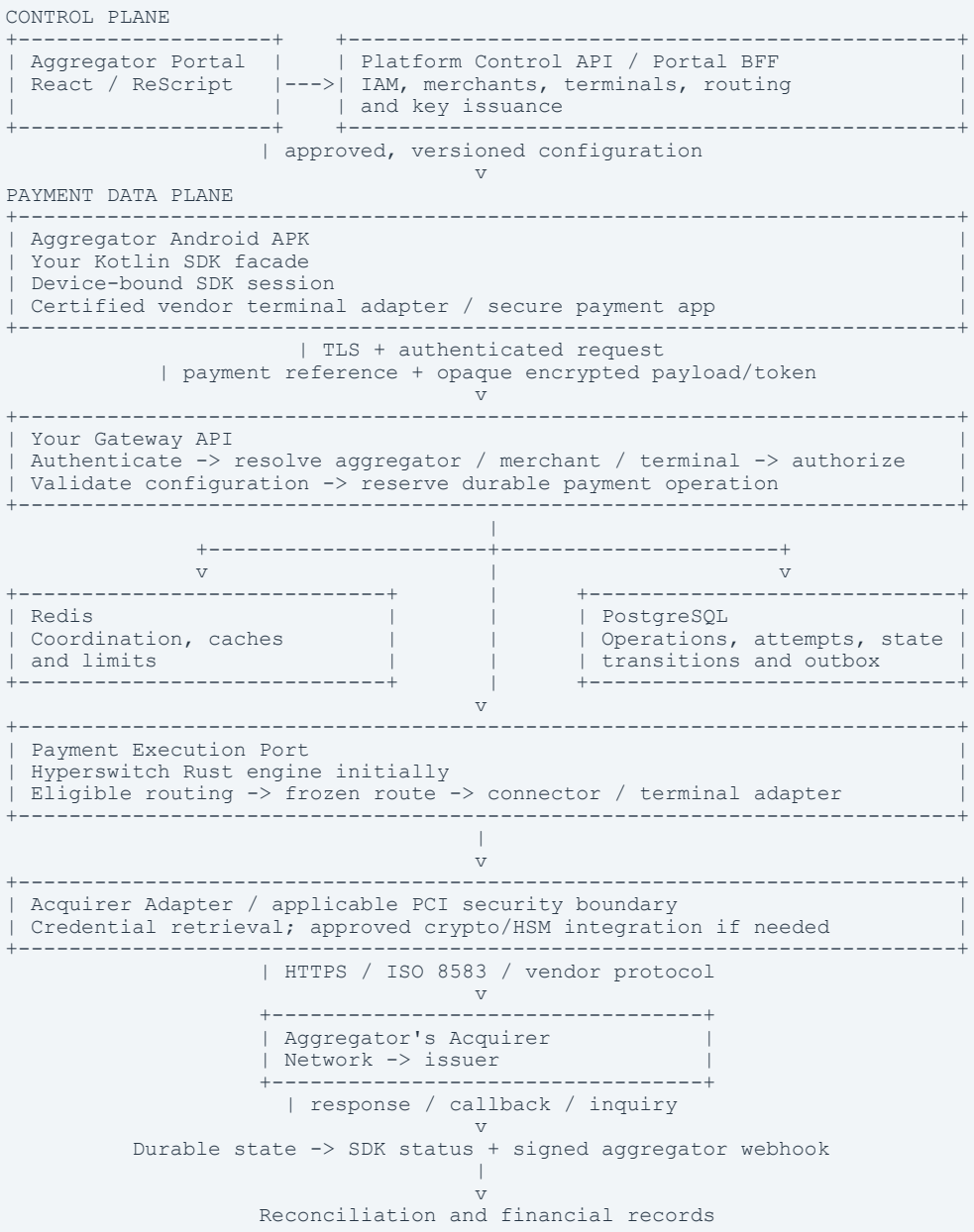
Decide the BusinessProfile mapping before introducing shared customer or vault features. In the inspected v1 schema, customers and stored payment methods are merchant-account scoped and do not have a BusinessProfile boundary. Mapping many legal merchants to profiles therefore does not automatically isolate their saved cards, customer records or mandates. See the customer model and payment method model.

For an initial terminal platform, use the requested mapping with explicit merchant ownership checks on every exposed resource. If each sub-merchant needs independent native credentials, vault boundaries or account-level behavior, evaluate Hyperswitch's existing Platform/Connected account model instead. The checkout already contains platform authentication and connected-merchant handling; that is an alternative topology, not another name for infrastructure tenancy. See the platform authentication implementation.

2. Architecture and service boundaries

Separate the control plane from payment execution. The recommended transaction path is:

TRANSACTION ARCHITECTURE



For terminal-encrypted traffic, the general gateway should preferably forward an opaque payload or token. It should not possess the corresponding decryption keys. The actual PCI boundary depends on the complete solution and its ability to affect payment security.

Use these domain boundaries:

Domain	Owns	Boundary
Identity and authorization	Users, memberships, operational keys, SDK sessions	Produces a trusted request context
Merchant management	Legal merchant records, onboarding, approval and suspension	Calls acquirer onboarding APIs through adapters
Terminal management	Enrollment, device keys, assignments, certification and bindings	Produces versioned terminal configuration
Processing configuration	Acquirer account references, routing policy, MCA projections	Controls eligible processing paths
Payment execution	Payment state, attempts, routing decisions, authorize/capture/refund/reversal	Exposes a language-neutral execution API
Webhooks and reconciliation	Inbound deduplication, outbound delivery, inquiries and settlement matching	Processes durable events
Portal and reporting	Tenant-scoped presentation and read models	Calls APIs; owns no payment truth

These are **logical service boundaries first**. Initially, some can share a deployment to reduce operational cost. Keep ownership and interfaces separate so extraction does not require reconstructing the domain.

For a future Kotlin transition:

- Publish your own versioned OpenAPI contract; use Protobuf/gRPC internally where useful.
- Keep Hyperswitch DTOs, connector enums and Diesel models inside the compatibility adapter.
- Use ports such as `PaymentExecutor`, `MerchantProvisioner`, `TerminalRegistry` and `CredentialProvider`.
- Give each domain sole ownership of its writable schema. Other services use APIs or events.
- Publish events through a transactional outbox; consumers use inbox deduplication.
- Replace implementations incrementally, with one authoritative writer per domain.
- Shadow only side-effect-free decisions. Never send the same live authorization through both implementations.

The platform payment workflow should own durable admission and policy. Hyperswitch should remain the initial authority for execution attempts and connector results. Maintain explicit mappings between the two state machines.

3. Aggregator Portal and access control

Make the Aggregator Portal an authenticated, tenant-scoped product. The upstream Control Center uses React and ReScript, with some TypeScript. Its configuration already exposes branding and endpoint settings, so start by adapting those surfaces. See the dashboard package configuration and runtime configuration.

Frontend changes should include:

- Aggregator branding resolved from a verified custom-domain mapping.
- Merchant onboarding and approval status.
- Terminal inventory, enrollment, assignment, suspension and processing bindings.
- Aggregator-owned acquirer connections and merchant-specific processing configuration.
- Payment, refund, dispute and reconciliation views scoped to the permitted merchant set.
- Operational-key and webhook management.
- Team invitations and scoped role management.

Treat branding as validated data: colors, approved logos, display names and support links. Avoid tenant-supplied executable JavaScript. Namespace frontend caches by environment and aggregator, and clear sensitive state on logout or context changes.

For browser authentication, use OIDC with a portal backend-for-frontend. Prefer secure, HttpOnly cookies with CSRF protection. API keys belong in aggregator servers, not browser storage.

On login:

1. Authenticate the user.
2. Load their active aggregator membership.
3. Resolve the corresponding organization and MerchantAccount.
4. Issue a session bound to that aggregator and environment.
5. Derive permissions and any permitted merchant subset from server-controlled membership data.

A suitable trusted context is:

JSON

```
{
  "subject": "user_123",
  "platform_id": "platform_live",
  "aggregator_id": "agg_A",
  "environment": "live",
  "permissions": ["merchants:create", "terminals:link", "payments:read"],
  "merchant_scope": "all",
  "authorization_version": 12
}
```

If users legitimately belong to several aggregators, switching context should produce a newly authorized session. A dropdown or request header must not independently change authority.

Use RBAC for actions and ownership checks for resources:

Role	Typical permissions
Aggregator administrator	Manage its merchants, users, keys, routing and terminal assignments
Onboarding operator	Create merchants and submit onboarding information
Operations operator	Inspect payments and perform specifically authorized operational actions
Developer	Manage integration settings, SDK sessions and permitted keys
Analyst	Read approved reports
Platform operator	Separate control-plane access with audited cross-aggregator operations

Restrict custom-role creation so users cannot grant permissions they do not possess.

Every backend operation must validate the full relationship:

CONTRACT / FLOW

```
principal → aggregator → merchant → terminal → processing binding
```

Apply that rule to individual reads, lists, search, exports, analytics, object-storage downloads, background jobs and websocket subscriptions. Hiding frontend menus is only presentation.

The native session model already carries merchant, organization, profile and tenant identifiers, providing useful integration points. It still needs your explicit aggregator membership and permission policy. See the native authentication token.

4. PostgreSQL tenancy and pooled connections

Enforce tenancy inside each database transaction, not only when borrowing a pooled connection. For every tenant-scoped unit of work:

SQL

```
BEGIN;

-- $1 comes from authenticated server context.
SELECT set_config('app.aggregator_id', $1, true);

-- Parameterized application queries run here.

COMMIT;
```

The final `true` makes the setting transaction-local. This avoids retaining one aggregator's context on a pooled connection after commit or rollback. PostgreSQL documents the corresponding lifetime in its SET reference.

Implement a required storage entry point:

CONTRACT / FLOW

```
with_aggregator_transaction(authenticated_context, operation)
```

It should:

- Reject missing tenant context.
- Acquire a connection and begin a transaction.
- Set trusted context before any tenant-scoped query.
- Expose scoped repositories rather than the raw pool.
- Commit or roll back before releasing the connection.

Use PostgreSQL RLS as defense in depth, with both `USING` and `WITH CHECK`. Runtime roles must be non-owner roles without `BYPASSRLS`; use `FORCE ROW LEVEL SECURITY` where applicable. PostgreSQL explains owner and superuser exceptions in its row security documentation.

Additional requirements:

- Include `aggregator_id` in indexes used by tenant-scoped queries and in relevant uniqueness constraints.
- Use composite foreign keys to prevent cross-aggregator relationships.
- Scope the **native Hyperswitch tables too**. Policies on your new registry do not protect native payments, refunds or customers.
- For native tables, enforce the resolved Hyperswitch MerchantAccount boundary; add explicit merchant/profile ownership where your public model needs it.
- Where denormalized tenant columns are added, backfill and validate their consistency before making them mandatory.
- Test accounts pools, payment pools, workers, replica reads and failed transactions.
- Namespace Redis entries, queues, exports, analytics filters and object-storage paths.

RLS based on a session setting protects against missing predicates. It does not independently protect against a fully compromised application that can set arbitrary context. Customers should never have database access. Stronger isolation can use separate database roles, databases or deployment cells.

The checkout supports both `PostgresOnly` and `RedisKv` payment storage paths. For the first production release, prefer authoritative PostgreSQL persistence for critical payment admission and state. RLS cannot protect records served exclusively from Redis; any KV path and drainer must receive equivalent isolation treatment. See the payment storage implementation.

5. Terminal identity and transaction routing

Authenticate the terminal before considering its routing fields. A submitted `terminal_id`, serial number or TID is a claim, not proof of identity.

Enrollment should establish:

CONTRACT / FLOW

```
aggregator
  └─ merchant
      └─ logical terminal deployment
          ├─ device public key / certificate
          ├─ approved APK package and signing identity
          ├─ terminal model and vendor payment application
          ├─ firmware / kernel / application versions
          ├─ activation and revocation state
          └─ permitted processing bindings
```

A processing binding should identify:

CONTRACT / FLOW

```
merchant + terminal deployment
  → acquirer account
  → profile-specific MCA projection
  → acquirer MID / TID
  → certification bundle
  → encryption/key domain
  → supported currencies and transaction types
  → binding/configuration version
```

On a charge:

1. Authenticate the SDK session and device proof.
2. Resolve aggregator, merchant and terminal from trusted server records.
3. Check all three are active and mutually related.
4. Validate amount, currency, operation, freshness and signed request fields.
5. Load eligible terminal bindings.
6. Intersect those bindings with aggregator routing policy.
7. Freeze the selected route, configuration and credential version on the attempt.
8. Execute through Hyperswitch or a dedicated terminal protocol adapter.

The SDK must not choose an arbitrary MCA, acquirer URL, credential reference or tenant header.

For terminal traffic, eligible routes are constrained by the certified integration and encryption domain. Ciphertext prepared for Acquirer A generally cannot be forwarded to Acquirer B. A fallback may require a new terminal interaction.

"Pre-certified terminal" also needs precise meaning. Hardware or kernel approval does not establish that a new gateway-to-acquirer integration is approved. EMVCo describes Level 3 testing as validation of the terminal's integration with its acceptance infrastructure. Track the approved combination and its change-control requirements. See EMV Level 3 testing.

Do not assume existing e-commerce connector requests support your terminal protocol. Add a typed terminal-payment contract and adapter where necessary. Define authorization, terminal completion, issuer scripts, reversal, inquiry and reconciliation behavior explicitly.

6. Distributed idempotency and payment safety

Use Redis for coordination and PostgreSQL for durable idempotency. A production payment design cannot safely promise duplicate prevention using Redis locks alone. Redis documents how asynchronous replication and failover can allow two clients to believe they own a lock. See Redis distributed lock guidance.

Use three layers:

Layer	Purpose
Redis	Coalesce concurrent requests, cache safe responses, enforce limits
PostgreSQL	Uniquely reserve operations and persist authoritative workflow state
Acquirer integration	Apply documented downstream duplicate controls and resolve ambiguous outcomes

Define the durable idempotency scope as:

CONTRACT / FLOW

```
environment + aggregator_id + merchant_id
+ operation_scope + idempotency_key
```

`operation_scope` distinguishes charge creation from capture, refund and reversal, including the target resource where appropriate. Do not include the calling operational key in the scope: rotating credentials must not permit the same operation again.

A safe algorithm is:

1. **Authenticate and authorize first.**
2. Compute a canonical request fingerprint covering immutable business fields and the stable payment-payload reference.
3. Attempt a short Redis lease using `SET ... NX PX`, with a random ownership value.
4. In a short PostgreSQL transaction, insert the operation using its unique idempotency constraint.
5. On conflict, load the existing operation. Same fingerprint and completed: return the stored safe result. Same fingerprint and running or uncertain: return its operation reference. Different fingerprint: return `409 idempotency_conflict`.
6. Commit the operation and dispatch reference before initiating the external financial action.
7. Claim the attempt using a conditional state transition.
8. Execute outside the database transaction.
9. Persist the result and an outbox event in a new short transaction.
10. Update the Redis response cache as an optimization.

A useful state model is:

CONTRACT / FLOW

```
ADMITTED → DISPATCHING → SUCCEEDED
                       → FAILED
                       → UNKNOWN → inquiry / callback / reconciliation
```

`UNKNOWN` means that a financial outcome may exist. It must not be treated as a failed charge.

Operational rules:

- Release or renew Redis leases using atomic ownership checks.
- Do not authorize a new charge merely because a lease expired.
- Use durable versions/fencing to reject stale database updates.
- Recognize that a database fence cannot stop an already-running external request.
- Reuse the original downstream transaction reference only according to the acquirer's documented duplicate semantics.
- A unique reference field alone is not proof that the acquirer deduplicates.
- If the worker crashes around dispatch, conservatively reconcile the original attempt before retrying.
- After an ambiguous timeout, do not automatically route to another acquirer.
- Deduplicate inbound acquirer notifications and enforce valid state transitions.
- Preserve durable duplicate protection beyond Redis TTL expiry.
- Define how SDK retries reuse the same encrypted request or stable payload reference; fresh randomized ciphertext must not accidentally redefine the operation.

Keep sensitive authentication payloads out of normal outbox records and replay caches. Dispatch events should contain secure references, not raw card/PIN data.

This design reduces contention through short transactions and indexed conditional writes. It cannot guarantee that database deadlocks never occur. Use consistent lock ordering, bounded lock waits and retries for safely repeatable database transactions; never repeat a financial side effect as part of a generic transaction retry.

Hyperswitch already has Redis-based API locking. Extend and integrate its behavior instead of assuming it is a complete durable idempotency contract. See the existing locking implementation.

For geographic resilience, give each aggregator or payment a home region and one authoritative writer. Start with controlled failover, not independent active writers sharing only eventually replicated idempotency data.

7. Three-tier credential hierarchy

Implement the three credential tiers as delegated authority. They should not be a chain of derivable secrets.

Tier	Holder	Authority
Platform master/control credentials	Isolated platform automation and operators	Provision aggregators and administer infrastructure
Aggregator operational keys	Aggregator backend	Authorized actions within one aggregator and environment
Ephemeral merchant/terminal tokens	Mobile SDK	Narrow actions for one merchant, terminal and preferably one payment session

For operational keys, store a lookup identifier, secure verifier/hash, aggregator, environment, scopes, expiry and revocation state. Support rotation with overlap, audit trails and separate keys for separate integrations.

Native API keys already have merchant ownership, hashing and expiry fields. Your finer scopes and delegated SDK issuance policy are platform extensions.

A mobile token could contain:

JSON

```
{
  "iss": "https://identity.gateway.example",
  "aud": "terminal-payments",
  "sub": "sdk-session-123",
  "aggregator_id": "agg_A",
  "merchant_id": "mer_B",
  "terminal_id": "term_C",
  "payment_session_id": "session_D",
  "scope": "payment:confirm payment:read",
  "currency": "MXN",
  "maximum_amount_minor": 12500,
  "configuration_version": 18,
  "cnf": {
    "jkt": "device-key-thumbprint"
  },
  "exp": 1792670700
}
```

The normal issuance flow is:

CONTRACT / FLOW

```
Aggregator backend authenticates with operational key
  → requests SDK session for an authorized merchant/terminal
  → gateway validates enrollment and requested payment constraints
  → gateway issues short-lived, device-bound token
  → SDK uses token with device proof
```

Do not allow arbitrary public minting from a publishable key. Publishable keys identify integrations; they are not operational authorization.

Use a standard sender-constrained token mechanism such as DPoP where suitable. DPoP provides token possession proofs; separately bind the transaction body and immutable business fields in the payment protocol. See RFC 9449.

The existing Hyperswitch `EphemeralKey` is customer-oriented and should not be assumed to provide this terminal-token hierarchy unchanged.

8. Android SDK white-labeling and APK integration

Make the Android SDK a thin, typed orchestration library around approved capture components. The upstream Android repository is part of the shared `hyperswitch-client-core` arrangement. The inspected main implementation also initializes React Native code, so the fork includes more than Kotlin files. See the Android repository, client-core setup and SDK initialization.

Create modules with narrow responsibilities:

CONTRACT / FLOW

```
gateway-sdk-api                 Stable Kotlin/Java-facing contracts
gateway-sdk-core                HTTP, sessions, retries, status handling
gateway-sdk-device              Enrollment and device-key operations
gateway-sdk-terminal-api        Vendor-neutral terminal interface
gateway-sdk-terminal-<vendor>   Certified vendor integration
gateway-sdk-ui                  Optional branded presentation
```

Reuse payment-sheet functionality where relevant. Keep terminal acquisition independent from checkout UI and shared JavaScript.

The inspected SDK already exposes custom backend, logging, asset, SDK configuration, confirm and Airborne endpoint overrides. Route each retained feature to your infrastructure and verify there is no fallback to upstream production endpoints. See the configuration model.

White-labeling work includes:

- Maven coordinates, package namespace and public API names.
- Resources, themes, logos, documentation and sample APKs.
- Manifest authorities, deep links and callback schemes.
- Logging, crash reporting, remote assets and update configuration.
- Signed releases, SBOMs and reproducible release inputs.
- Preservation of applicable license and attribution notices.

Use runtime branding/configuration where possible rather than maintaining a code fork per aggregator.

Capture configuration in three categories:

Category	Examples	Authority
Observed device facts	Model, firmware, payment-app version, capabilities	Vendor/device evidence
Server assignments	Merchant, MID/TID, processing bindings, configuration version	Terminal registry
Transaction data	Amount, currency, order reference, entry mode, secure payload	Validated payment session and certified capture flow

The client may report observations. It must not establish its own merchant assignment or acquirer configuration.

Use hardware-backed Android Keystore keys where supported. Validate attestation on a trusted server, including the challenge, trust chain and revocation state. Android attestation is additional device evidence; it does not itself establish terminal payment certification. See Android key attestation.

Secure capture and encryption

Keep raw payment capture inside the approved security boundary. For certified physical or smart terminals, prefer:

CONTRACT / FLOW

```
Card/PIN
  → approved terminal secure component
  → vendor/acquirer-approved encryption
  → opaque encrypted result or token
  → gateway forwarding
  → authorized decryption endpoint
```

Do not design your general SDK to read clear PAN, track data or PIN blocks and then apply custom RSA encryption.

Where an approved integration requires application-layer public-key encryption:

- Use a vetted envelope format such as JWE and an explicitly allowed algorithm suite.
- Fetch recipient keys from authenticated, versioned configuration.
- Validate `kid`, key purpose, environment and expiry.
- Bind amount, currency, merchant, terminal, operation and session to the authenticated envelope.
- Keep private decryption keys outside the APK and general gateway.
- Retain TLS; message encryption does not authenticate the transport endpoint.

JWE defines the envelope structure, not a PCI approval or terminal key-management system. Terminal PIN/data encryption and HSM key management must follow the selected approved solution. See RFC 7516.

PCI SSC states that PTS-approved terminals alone do not guarantee scope reduction; validated P2PE solutions can facilitate it. If your gateway decrypts payment data, plan for the corresponding cardholder-data environment. Confirm the actual service-provider boundary against the deployed solution. See PCI terminal assessment guidance.

For ordinary Android phones accepting NFC or PIN, use an appropriate validated mobile acceptance solution rather than treating the checkout SDK as a SoftPOS stack. See PCI MPoC.

APK communication and recovery

Use local calls for an embedded SDK and secured IPC only where another app is involved.

- Inside the aggregator APK, prefer Kotlin APIs/coroutines and lifecycle-aware callbacks.
- Keep internal services non-exported.
- For another payment app, use an explicit component and the vendor's documented service/Activity Result contract.
- For exported Binder/AIDL services, verify caller identity and authorization on each request.
- Use signature permissions when applications share the appropriate signing identity. Different aggregator signers require a deliberate partner trust model.
- Pass operation IDs and opaque references through IPC; avoid broadcasts and sensitive intent extras.

Android documents the risks of exposed components and the same-certificate behavior of signature permissions in its component access-control guidance.

The SDK must represent `pending` and `unknown` results. After process death or connectivity loss, recover status using the original operation ID. A UI cancellation does not prove that an authorization was cancelled.

Persist only the minimum recovery metadata. Do not use generic background retries to submit new authorizations. Sensitive authentication data cannot be retained after authorization even when encrypted; this includes relevant encrypted queues and diagnostic captures. See PCI storage guidance.

9. Database schema draft

Use an additive relational schema with explicit ownership. The following is a logical schema draft, not a migration against the current Diesel schema. `hs_*` columns are compatibility references; native identifiers remain internal.

The relationships are:

CONTRACT / FLOW

```
Platform
  └─ Aggregator
      ├─ Merchant → Hyperswitch BusinessProfile
      │   ├─ Profile Connector → Hyperswitch MCA
      │   └─ Terminal Deployment
      │       └─ Terminal Binding → Profile Connector
      ├─ Acquirer Account → versioned credential reference
      └─ Payment Operation
          └─ Acquirer Attempt
```

SQL

```
CREATE SCHEMA gateway;

CREATE TABLE gateway.platforms (
    platform_id uuid PRIMARY KEY,
    environment text NOT NULL
        CHECK (environment IN ('sandbox', 'live')),
    hs_tenant_id text NOT NULL,
    UNIQUE (environment, hs_tenant_id)
);
```

SQL - CONTINUED

```
CREATE TABLE gateway.aggregators (
    aggregator_id uuid PRIMARY KEY,
    platform_id uuid NOT NULL
        REFERENCES gateway.platforms(platform_id),
    hs_organization_id text NOT NULL,
    hs_merchant_account_id text NOT NULL,
    status text NOT NULL
        CHECK (status IN ('provisioning', 'active', 'suspended', 'closed')),
    UNIQUE (platform_id, hs_organization_id),
    UNIQUE (platform_id, hs_merchant_account_id)
);

CREATE TABLE gateway.merchants (
    aggregator_id uuid NOT NULL
        REFERENCES gateway.aggregators(aggregator_id),
    merchant_id uuid NOT NULL,
    external_reference text NOT NULL,
    legal_entity_ref text NOT NULL,
    hs_profile_id text, -- null until provisioned
    status text NOT NULL
        CHECK (status IN (
            'provisioning', 'pending_review', 'active', 'suspended', 'closed'
        )),
    created_at timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (aggregator_id, merchant_id),
    UNIQUE (aggregator_id, external_reference),
    UNIQUE (aggregator_id, hs_profile_id)
);

CREATE TABLE gateway.acquirer_accounts (
    aggregator_id uuid NOT NULL
        REFERENCES gateway.aggregators(aggregator_id),
    acquirer_account_id uuid NOT NULL,
    connector_code text NOT NULL,
    credential_version_ref text NOT NULL, -- secret store reference
    status text NOT NULL,
    PRIMARY KEY (aggregator_id, acquirer_account_id)
);

CREATE TABLE gateway.profile_connectors (
    aggregator_id uuid NOT NULL,
    merchant_id uuid NOT NULL,
    connector_id uuid NOT NULL,
    acquirer_account_id uuid NOT NULL,
    hs_mca_id text,
    config_version bigint NOT NULL,
    status text NOT NULL,
    PRIMARY KEY (aggregator_id, merchant_id, connector_id),
    UNIQUE (aggregator_id, hs_mca_id),
    FOREIGN KEY (aggregator_id, merchant_id)
        REFERENCES gateway.merchants(aggregator_id, merchant_id),
    FOREIGN KEY (aggregator_id, acquirer_account_id)
        REFERENCES gateway.acquirer_accounts(
            aggregator_id, acquirer_account_id
        )
);
```

SQL - CONTINUED

```
CREATE TABLE gateway.terminals (
    aggregator_id uuid NOT NULL,
    merchant_id uuid NOT NULL,
    terminal_id uuid NOT NULL,
    vendor text NOT NULL,
    model text NOT NULL,
    device_key_thumbprint text NOT NULL,
    approved_app_identity_ref text NOT NULL,
    configuration_version bigint NOT NULL,
    status text NOT NULL,
    PRIMARY KEY (aggregator_id, terminal_id),
    UNIQUE (aggregator_id, merchant_id, terminal_id),
    FOREIGN KEY (aggregator_id, merchant_id)
        REFERENCES gateway.merchants(aggregator_id, merchant_id)
);

CREATE TABLE gateway.terminal_bindings (
    aggregator_id uuid NOT NULL,
    binding_id uuid NOT NULL,
    merchant_id uuid NOT NULL,
    terminal_id uuid NOT NULL,
    connector_id uuid NOT NULL,
    acquirer_mid text NOT NULL,
    acquirer_tid text NOT NULL,
    certification_bundle_ref text NOT NULL,
    encryption_domain_ref text NOT NULL,
    binding_version bigint NOT NULL,
    active boolean NOT NULL DEFAULT false,
    PRIMARY KEY (aggregator_id, binding_id),
    FOREIGN KEY (aggregator_id, merchant_id, terminal_id)
        REFERENCES gateway.terminals(
            aggregator_id, merchant_id, terminal_id
        ),
    FOREIGN KEY (aggregator_id, merchant_id, connector_id)
        REFERENCES gateway.profile_connectors(
            aggregator_id, merchant_id, connector_id
        )
);

CREATE UNIQUE INDEX active_terminal_connector_binding
    ON gateway.terminal_bindings(
        aggregator_id, terminal_id, connector_id
    )
    WHERE active;
```

SQL - CONTINUED

```
CREATE TABLE gateway.payment_operations (
    aggregator_id uuid NOT NULL,
    operation_id uuid NOT NULL,
    merchant_id uuid NOT NULL,
    operation_scope text NOT NULL,
    idempotency_key_hash bytea NOT NULL,
    canonical_request_hash bytea NOT NULL,
    state text NOT NULL
        CHECK (state IN (
            'admitted', 'dispatching', 'succeeded', 'failed', 'unknown'
        )),
    version bigint NOT NULL DEFAULT 0,
    route_snapshot jsonb, -- identifiers/config versions; no secrets or SAD
    safe_result jsonb,
    created_at timestamptz NOT NULL DEFAULT now(),
    updated_at timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (aggregator_id, operation_id),
    FOREIGN KEY (aggregator_id, merchant_id)
        REFERENCES gateway.merchants(aggregator_id, merchant_id),
    UNIQUE (
        aggregator_id, merchant_id,
        operation_scope, idempotency_key_hash
    )
);

CREATE TABLE gateway.acquirer_attempts (
    aggregator_id uuid NOT NULL,
    attempt_id uuid NOT NULL,
    operation_id uuid NOT NULL,
    acquirer_account_id uuid NOT NULL,
    acquirer_request_reference text NOT NULL,
    hs_payment_id text,
    state text NOT NULL,
    PRIMARY KEY (aggregator_id, attempt_id),
    FOREIGN KEY (aggregator_id, operation_id)
        REFERENCES gateway.payment_operations(
            aggregator_id, operation_id
        ),
    FOREIGN KEY (aggregator_id, acquirer_account_id)
        REFERENCES gateway.acquirer_accounts(
            aggregator_id, acquirer_account_id
        ),
    UNIQUE (
        aggregator_id, acquirer_account_id,
        acquirer_request_reference
    )
);
```

The composite references prevent, for example, assigning an aggregator's terminal to another aggregator's merchant.

For RLS, a minimal pattern is:

```sql
CREATE FUNCTION gateway.current_aggregator_id()
RETURNS uuid
LANGUAGE sql
STABLE
AS $$
    SELECT NULLIF(
        current_setting('app.aggregator_id', true), ''
    )::uuid
$$;

ALTER TABLE gateway.merchants ENABLE ROW LEVEL SECURITY;
ALTER TABLE gateway.merchants FORCE ROW LEVEL SECURITY;

-- Provision gateway_app separately as a non-owner runtime role.
CREATE POLICY merchant_aggregator_isolation
ON gateway.merchants
TO gateway_app
USING (
    aggregator_id = gateway.current_aggregator_id()
)
WITH CHECK (
    aggregator_id = gateway.current_aggregator_id()
);
```

Apply equivalent policies to all tenant-owned tables, with service-specific grants. Missing context should deny access.

The complete production model also needs memberships, operational-key verifiers, SDK sessions, certification records, onboarding jobs, webhook inbox/outbox, delivery attempts and audit records. Keep those within their owning domains.

MCA projection requires special care: native MCAs are profile-associated. An aggregator-owned acquirer credential may therefore produce multiple merchant/profile-specific MCA projections. Store the common credential as a controlled secret reference; provision only the configuration each merchant is authorized to use.

Native-object creation and your registry updates form a provisioning workflow, not a cross-service SQL transaction. Make every step idempotent, record its result and reconcile incomplete projections.

10. API architecture contract

Expose a stable merchant onboarding API owned by your platform. The requested endpoint should be a public platform contract, not a renamed native admin endpoint:

```http
POST /v1/aggregators/{aggregator_id}/merchants
Authorization: Bearer <aggregator-operational-key>
Idempotency-Key: merchant-onboarding-8472
Content-Type: application/json
```

Example:

JSON

```
{
  "external_reference": "merchant-8472",
  "legal_entity": {
    "registered_name": "Comercial Ejemplo SA de CV",
    "trading_name": "Ejemplo Centro",
    "country": "MX",
    "business_type": "company",
    "merchant_category_code": "5411",
    "kyb_document_set_id": "documents-8472"
  },
  "contact": {
    "name": "Operations Team",
    "email": "operations@example-merchant.com"
  },
  "business_address": {
    "line1": "Av. Ejemplo 100",
    "city": "Ciudad de Mexico",
    "postal_code": "06000",
    "country": "MX"
  },
  "processing": {
    "currencies": ["MXN"],
    "channels": ["card_present"],
    "requested_acquirer_accounts": [
      {
        "acquirer_account_id": "acqacct_primary",
        "settlement_profile_id": "settlement_approved_01"
      }
    ],
    "routing_policy_id": "routing_retail_mx_v3"
  },
  "metadata": {
    "aggregator_store_reference": "store-019"
  }
}
```

Identifiers above are shortened for readability.

The server must:

1. Match the path aggregator to the authenticated key's authority.
2. Require `merchants:create`.
3. Validate ownership of document, settlement, routing and acquirer references.
4. Reserve the idempotent onboarding operation.
5. Create the canonical merchant record.
6. Provision its BusinessProfile under the aggregator's MerchantAccount.
7. Submit required acquirer enrollment through the relevant adapter.
8. Create profile-specific connector projections as approvals permit.
9. Activate processing only after all required checks succeed.

Return an asynchronous resource:

HTTP

```
HTTP/1.1 202 Accepted
Location: /v1/aggregators/agg_A/operations/onboard_123
```

JSON

```
{
  "operation_id": "onboard_123",
  "merchant": {
    "id": "mer_B",
    "external_reference": "merchant-8472",
    "status": "pending_review"
  },
  "processing_status": "not_enabled"
}
```

Creating a profile must not imply acquirer acceptance or permission to process.

Recommended related endpoints:

Endpoint	Purpose
`GET /v1/aggregators/{a}/merchants/{m}`	Merchant lifecycle and processing status
`POST /v1/aggregators/{a}/terminal-enrollments`	Begin controlled device enrollment
`POST /v1/aggregators/{a}/merchants/{m}/terminal-bindings`	Link an enrolled terminal to an approved processing configuration
`POST /v1/aggregators/{a}/sdk-sessions`	Issue constrained SDK authority
`POST /v1/charges`	Submit a charge; derive ownership from authenticated context
`GET /v1/operations/{id}`	Recover operation status with ownership checks
`POST /v1/payments/{id}/captures`	Capture under original payment constraints
`POST /v1/payments/{id}/refunds`	Create an independently idempotent refund

For SDK payment submission, use integer minor-unit amounts and typed secure payloads:

JSON

```
{
  "payment_session_id": "session_D",
  "merchant_reference": "order-20261022-001",
  "amount_minor": 12500,
  "currency": "MXN",
  "terminal_configuration_version": 18,
  "payment_payload": {
    "type": "acquirer_encrypted",
    "format": "approved-vendor-format-v1",
    "reference": "secure-payload-reference"
  }
}
```

The session determines the aggregator, merchant and terminal. Any identifiers repeated in the body must match; the body cannot expand authority.

Define consistent errors:

- `401`: invalid or expired authentication.
- `403` or ownership-safe `404`: unauthorized resource access.
- `409`: changed payload under an existing idempotency key, or conflicting state.
- `422`: invalid configuration or unsupported operation.
- `429`: scoped quota exceeded.
- `202`: accepted operation whose financial outcome remains pending.

Sign outbound webhooks, include event ID and resource version, support replay detection and retries, and assume delivery can be duplicated or reordered.

11. Engineering roadmap and production gates

Deliver the implementation through four phases with measurable exit criteria.

Phase	Engineering work	Exit criteria
1. Environment & Fork Isolation	Fork and pin core, dashboard, Android and required client-core dependencies. Inventory licenses, upstream endpoints, telemetry and remote assets. Establish your domains, signing, CI, secret management and separate sandbox/live infrastructure. Identify the first terminal/acquirer integration and its approval requirements.	Reproducible builds; baseline native payment flow works; retained SDK features use approved infrastructure; dependency/version manifest exists.

Phase	Engineering work	Exit criteria
2. Database & Tenancy Refactoring	Implement canonical aggregator/merchant records and compatibility mappings. Introduce trusted request context, scoped repositories, transaction-local RLS, composite constraints and cache namespaces. Implement durable payment admission, idempotency and event delivery. Backfill using expand/validate/contract migrations.	Cross-aggregator access tests fail closed across APIs, SQL, caches, analytics and workers. Concurrent duplicates and Redis failure do not create a second admitted operation. Pool reuse and rollback tests pass.
3. Terminal & SDK Customization	Build enrollment, device-bound sessions, terminal bindings, certification/version checks and vendor adapters. White-label SDK artifacts and connection points. Implement secure payload transport, IPC, process recovery, terminal completion, inquiry and reversal.	One complete approved terminal-to-acquirer path works. Unbound/revoked terminals fail. App death and network loss recover by operation ID. Acquirer timeout/crash tests preserve uncertainty without blind resubmission.
4. Aggregator Console & API Exposure	Deliver branded portal, merchant onboarding, terminal inventory/linking, scoped roles, operational keys, SDK-session APIs, reports and signed webhooks. Integrate approval workflows and audit records.	An aggregator can onboard, activate, bind and transact for its own merchant; lifecycle and permissions work end to end; settlement reconciliation matches the pilot flow.

Although the portal arrives in Phase 4, implement its API contracts and authorization foundations in Phase 2 so SDK work uses the final boundaries.

Before live rollout, require:

- **Financial correctness:** duplicate delivery, worker crashes, ambiguous outcomes, capture/refund limits, reversal and settlement mismatch handling.
- **Isolation:** negative tests against another aggregator's IDs, exports, analytics, credentials, SDK sessions and cached objects.
- **Certification:** acceptance of the actual terminal/application/gateway/acquirer combination and documented change control.
- **Recovery:** tested backups, restore, home-region failover and key rotation.
- **Operations:** latency and availability objectives, queue-age alerts, reconciliation-age alerts, per-aggregator limits and emergency disable controls.
- **Observability:** traceable operation/attempt IDs and redacted logs; no payment payloads in generic logs or crash reports.

If acquirers settle directly, reconciliation and an immutable transaction journal may be sufficient initially. If your platform later owns balances, fees, reserves or payouts, introduce a dedicated double-entry ledger with its own accounting invariants.

The first production slice should be **one aggregator, one merchant, one terminal family and one acquirer**, exercising onboarding through settlement reconciliation. Expand networks and terminal models only after that full lifecycle is reliable.

Reference index

All references below are clickable. Local repository references have been converted to the pinned public core commit for portability.

01. tenant configuration model
https://github.com/juspay/hyperswitch/blob/c5ecb434b419d2c473cf2882a3866de9146abdec/crates/hyperswitch_interfaces/src/configs.rs#L10

02. profile model
https://github.com/juspay/hyperswitch/blob/c5ecb434b419d2c473cf2882a3866de9146abdec/crates/diesel_models/src/business_profile.rs#L32

03. MCA model
https://github.com/juspay/hyperswitch/blob/c5ecb434b419d2c473cf2882a3866de9146abdec/crates/diesel_models/src/merchant_connector_account.rs#L28

04. customer model
https://github.com/juspay/hyperswitch/blob/c5ecb434b419d2c473cf2882a3866de9146abdec/crates/diesel_models/src/customers.rs#L152

05. payment method model
https://github.com/juspay/hyperswitch/blob/c5ecb434b419d2c473cf2882a3866de9146abdec/crates/diesel_models/src/payment_method.rs#L27

06. platform authentication implementation
https://github.com/juspay/hyperswitch/blob/c5ecb434b419d2c473cf2882a3866de9146abdec/crates/router/src/services/authentication.rs#L3638

07. dashboard package configuration
https://github.com/juspay/hyperswitch-control-center/blob/main/package.json

08. runtime configuration
https://github.com/juspay/hyperswitch-control-center/blob/main/config/config.toml

09. native authentication token
https://github.com/juspay/hyperswitch/blob/c5ecb434b419d2c473cf2882a3866de9146abdec/crates/router/src/services/authentication.rs#L325

10. SET reference
https://www.postgresql.org/docs/current/sql-set.html

11. row security documentation
https://www.postgresql.org/docs/current/ddl-rowsecurity.html

12. payment storage implementation
https://github.com/juspay/hyperswitch/blob/c5ecb434b419d2c473cf2882a3866de9146abdec/crates/storage_impl/src/payments/payment_intent.rs#L119

13. EMV Level 3 testing
https://www.emvco.com/knowledge-hub/what-is-level-3-terminal-integration-testing/

14. Redis distributed lock guidance
https://redis.io/docs/latest/develop/clients/patterns/distributed-locks/

15. existing locking implementation
https://github.com/juspay/hyperswitch/blob/c5ecb434b419d2c473cf2882a3866de9146abdec/crates/router/src/core/api_locking.rs#L43

16. RFC 9449
https://www.rfc-editor.org/rfc/rfc9449.html

17. Android repository
https://github.com/juspay/hyperswitch-sdk-android/blob/main/README.md

18. client-core setup
https://github.com/juspay/hyperswitch-client-core/blob/main/README.md

19. SDK initialization
https://github.com/juspay/hyperswitch-sdk-android/blob/f77ec2f6883e324484e2f7cf6ce6d0939aa8e533/app/src/main/kotlin/io/hyperswitch/sdk/Hyperswitch.kt

20. configuration model
https://github.com/juspay/hyperswitch-sdk-android/blob/f77ec2f6883e324484e2f7cf6ce6d0939aa8e533/hyperswitch-sdk-android-api/src/main/kotlin/io/hyperswitch/model/Model.kt

21. Android key attestation
https://developer.android.com/privacy-and-security/security-key-attestation

22. RFC 7516
https://www.rfc-editor.org/rfc/rfc7516.html

23. PCI terminal assessment guidance
https://www.pcisecuritystandards.org/faqs/1301/

24. PCI MPoC
https://www.pcisecuritystandards.org/standards/mobile-payments-on-cots-mpoc/

25. component access-control guidance
https://developer.android.com/privacy-and-security/risks/access-control-to-exported-components

26. PCI storage guidance
https://www.pcisecuritystandards.org/faqs/1154/